UNIVERSITY OF RWANDA

College Of Science and Technology

Computer Engineering

Year *3*

Gako Campus

Mobile Computing

Reg n°: 223026685

MOBILE COMPUTING ASSIGNMENT

QUESTIONS:

Q1. Explain briefly the following state management used in Flutter application development 3
Q2. Draw a table explaining the situation where each state management is applicable.....4
Q3. Explain in detail the key steps on how Provider is used 5
1. Adding Dependency.....5
2. Creating a State Class.....5
3. Providing the State 6
4. Accessing the State 7
There are three ways to access the state in your widgets.....7
Method 1: Using Provider.of 8
Method 2: Using Consumer (Recommended for rebuild optimization).....9
Method 3: Using Consumer for specific parts.....9
5. Updating the State 10
6. How UI Rebuild Happens 12

Q1. Explain briefly the following state management used in Flutter application development

Q1. Explain briefly the following State Management Used In flutter Application Development

* Provider : Is a wrapper Around Inherited Widget. that makes States Management Easier. It allows Some one to Expose and Consume Data throughout the Widget Tree Using " provider.of<T>(Context) or Consumer Widgets. It's simple, testable and recomended by google for small to medium Application

* Riverpod : Is an Improved Version of provider that Compiles Safely It doesn't depend on flutter's Widget Tree, making it more Robust. It offers better Error handling, no Build Context dependency and work well with Both Synchronous and Asynchoronous Code.

* Bloc (Business logic Component) : Bloc Uses Streams and Sinks to Manage State. It separates business logic from UI Using Events and States. The UI dispatches Events and Bloc Emits New states in Response. It is predictable, testable and ideal for Complex Application with strict Architecture.

* Getx : Is a micro-framework that Combine state Managment, dependency Injection and route Managment. It offers reactive State Management with minimal Boilerplate, High performance and build-in navigation.

It is popular for rapid development and small to medium Project

Q2. Draw a table explaining the situation where each state management is applicable

Q2. Draw a table Explaining the situation where Each State Management is Applicable.

Situation	Recommended State Management	Explanation
* Small Application	. Set State( )	, Simple and built-in method in flutter , Easy to use when the App has few screens and minimal logic
* Medium Application	. provider	, Helps, Manage, shared data between Multiple widgets and keeps the Code organised
* Large / Enterprise Application	. Bloc (Business logic Component)	, Separates business logic from UI making the App Scalable and Easier to maintain.
* Team project	. Bloc / Riverpod	, provide clear structure so Multiple developers can work on different parts without Conflict.
* Fast development	. provider	, Quick to Implement and Requires less boilerplate Code. Good for Rapid prototyping
* Strict Architecture Requirement	. Bloc	, Enforce Clear separations of Concerns and Predictable data flow

Addition Information.

Application Type / Requirements	provider	Riverpod	Bloc	Get X
* Small Application	Best choice	Good	Over kill	Good
* Medium Application	Good	Best choice	possible	Good
* Large / Enterprise Application	Limitted	Good	Best choice	less structured
* Team project	Good	Good	Best choice	less structured
* fast development	Good	Good	Time Consuming	Best
* structured Requirement	Limitted	Good	Best choice	Less structured

Q3. Explain in detail the key steps on how Provider is used. (Computers can be used to copy and paste code snippets)

1. Adding dependency
2. Creating a state class
3. Providing the state
4. Accessing the state
5. Updating the state
6. How UI rebuild happens

Solution

1. Adding Dependency

First, add the provider package to your pubspec.yaml file

code snippets

```yaml
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.1  # Use the latest version
```

Then run flutter pub get in the terminal to install the package

2. Creating a State Class

Create a class that extends ChangeNotifier to manage your state:

```dart
// models/counter_model.dart
import 'package:flutter/material.dart';

class CounterModel extends ChangeNotifier {
  int _count = 0;

  int get count => _count;

  void increment() {
    _count++;
    notifyListeners(); // This triggers UI rebuild
  }

  void decrement() {
    _count--;
    notifyListeners(); // This triggers UI rebuild
  }

  void reset() {
    _count = 0;
    notifyListeners(); // This triggers UI rebuild
  }
}
```

The class extends ChangeNotifier which provides change notification capabilities

Explanation

1. Private variable _count stores the state
2. Getter count exposes the value safely
3. Methods modify the state and call notifyListeners() to update the UI

3. Providing the State

Wrap your app or a part of it with ChangeNotifierProvider:

```dart
// main.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'models/counter_model.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => CounterModel(),
      child: MaterialApp(
        title: 'Provider Demo',
        theme: ThemeData(primarySwatch: Colors.blue),
        home: HomeScreen(),
      ),
    );
  }
}
```

Explanation

1. ChangeNotifierProvider makes the state available to all descendant widgets
2. The create function instantiates the model once
3. MultiProvider helps when you have multiple state classes

4. Accessing the State

There are three ways to access the state in your widgets:

Method 1: Using Provider.of

```
// screens/home_screen.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/counter_model.dart';

class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // Access the model
    final counterModel = Provider.of<CounterModel>(context);

    return Scaffold(
      appBar: AppBar(title: Text('Provider Demo')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text('Counter Value:', style: TextStyle(fontSize: 20)),
            Text(
              '${counterModel.count}',
              style: TextStyle(fontSize: 40, fontWeight: FontWeight.bold),
            ),
          ],
        ),
      ),
    );
  }
}
```

Method 2: Using Consumer (Recommended for rebuild optimization)

```
class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Provider Demo')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text('Counter Value:', style: TextStyle(fontSize: 20)),

            // Only this part rebuilds when count changes
            Consumer<CounterModel>(
              builder: (context, counterModel, child) {
                return Text(
                  '${counterModel.count}',
                  style: TextStyle(fontSize: 40, fontWeight: FontWeight.bold),
                );
              },
            ),
          ],
        ),
      ),
    );
  }
}
```

Method 3: Using Consumer for specific parts

```dart
// Access only the value you need
final count = context.select<CounterModel, int>((model) => model.count);
```

Explanation

1. Provider.of<T>(context) gives direct access but rebuilds the whole widget
2. Consumer<T> rebuilds only its child when the state changes
3. context.select() rebuilds only when the selected value changes

5. Updating the State

Call methods on the model to update state:

```
// Add buttons to update state
class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Provider Demo')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text('Counter Value:', style: TextStyle(fontSize: 20)),
            Consumer<CounterModel>(
              builder: (context, counterModel, child) {
                return Text(
                  '${counterModel.count}',
                  style: TextStyle(fontSize: 40, fontWeight: FontWeight.bold),
                );
              },
            ),
            SizedBox(height: 20),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                // Access model directly in onPressed
                Consumer<CounterModel>(
                  builder: (context, counterModel, child) {
                    return ElevatedButton(
                      onPressed: counterModel.increment,
                      child: Text('Increment'),
```

```
                    );
                  },
                ),
                SizedBox(width: 10),
                Consumer<CounterModel>(
                  builder: (context, counterModel, child) {
                    return ElevatedButton(
                      onPressed: counterModel.decrement,
                      child: Text('Decrement'),
                    );
                  },
                ),
                SizedBox(width: 10),
                Consumer<CounterModel>(
                  builder: (context, counterModel, child) {
                    return ElevatedButton(
                      onPressed: counterModel.reset,
                      child: Text('Reset'),
                    );
                  },
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

Alternative: Using Provider.of with listen: false

```dart
// In methods where you don't need rebuild
void _handleIncrement(BuildContext context) {
  // listen: false prevents unnecessary rebuilds
  Provider.of<CounterModel>(context, listen: false).increment();
}
```

Explanation

1. Calling methods on the model updates the internal state
2. Each method calls notifyListeners() which triggers UI updates
3. listen: false is used when you don't need the widget to rebuild

6. How UI Rebuild Happens

The rebuild mechanism in Provider follows this flow:

```dart
// Step-by-step rebuild process

// 1. Model extends ChangeNotifier
class CartModel extends ChangeNotifier {
  List<Item> _items = [];

  void addItem(Item item) {
    _items.add(item);

    // 3. notifyListeners() is called
    notifyListeners();
  }
}

// 2. Some action triggers state change
ElevatedButton(
  onPressed: () {
    // This calls addItem which modifies state
    context.read<CartModel>().addItem(Item(name: 'Product'));
  },
  child: Text('Add to Cart'),
),

// 4. Consumer widgets rebuild automatically
Consumer<CartModel>(
  builder: (context, cartModel, child) {
    // This rebuilds when notifyListeners() is called
    return Text('Total items: ${cartModel.items.length}');
  },
)

// 5. Provider.of with listen: true also rebuilds
final cartModel = Provider.of<CartModel>(context); // Listens for changes
```

How the rebuild mechanism works internally

1. ChangeNotifier maintains a list of listeners
2. When notifyListeners() is called, it iterates through all listeners
3. Consumer widgets register themselves as listeners when created
4. The listener callback triggers setState() on the Consumer's internal State object
5. Only the Consumer's builder method runs, rebuilding its child widgets
6. Other parts of the widget tree remain unchanged

Summary Table of Provider Concepts

Concept	Purpose	Key Method/Widget
ChangeNotifier	Creates observable state class	notifyListeners()
ChangeNotifierProvider	Provides state to widget tree	create: parameter
Provider.of<T>(context)	Accesses state (with/without listen)	listen: true/false
Consumer<T>	Rebuilds only specific part	builder: callback
context.read<T>()	One-time access without listening	Use in callbacks
context.watch<T>()	Listens and rebuilds	Alternative to Consumer
MultiProvider	Provides multiple states	List of providers

----------End----------